PES-VCS
Version Control System

Student Name	Ashith Rao
SRN	PES1UG24CS559
Course	Operating Systems — Unit 4
Date	April 17, 2026
Platform	Ubuntu 22.04 / gcc

PHASE 1

Object Storage Foundation

Content-addressable store · SHA-256 hashing · Atomic writes · Directory sharding

Screenshot 1A — Unit Test Results

The test_objects program verifies three core properties of the object store: correct blob storage and retrieval, deduplication (same content → same hash → stored once), and SHA-256 integrity checking (detects tampered objects). All three tests pass, confirming that **object_write** and **object_read** are correctly implemented.

```
no changes added to commit (use "git add" and/or "git commit -a")
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ gedit object.c
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ git add object.c
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ git commit -m "Phase 1: Update object file permissions to 0644 to satisfy integrity test requirements"
[main b0293e7] Phase 1: Update object file permissions to 0644 to satisfy integrity test requirements
 1 file changed, 134 insertions(+), 97 deletions(-)
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$
```

Screenshot 1A: ./test_objects output — All Phase 1 tests passed (blob storage, deduplication, integrity check)

Screenshot 1B — Sharded Directory Structure

Objects are stored under .pes/objects/XX/YYY... where *XX* is the first two hex characters of the SHA-256 hash. This sharding prevents any single directory from accumulating thousands of entries, mirroring Git's real on-disk layout.

```
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ find .pes/objects -type f | sort
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/0a/8328b9912919497a605be0bdca795c47b30d0ab39864c58d88eba22103c573
.pes/objects/0a/ee4d975e3db22615631a09c457786a488968b6948ef756cbef4ec2ea09cbb6
.pes/objects/26/1e9d0bd1e8a73c2f3d706136d81de27add0e563f2471f0efda040daf495c93
.pes/objects/59/7ee0b9700690fe05fc7dd10c5fef15473d2326ce525df193d8009405d29444
.pes/objects/63/a5c732f5e624ae4bf855f234845e9894c8311486e6b4acc8c17f0a0eb23de9
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/74/3c00726e48a1af9ed5f8ce4f7d148a2c96a279ed3be4c18197a2817325c920
.pes/objects/a1/0d83f2ac5aa39f5af1ba8669ba035ad4369ad12c70f8f2a9ddcea6ae197153
.pes/objects/af/609ab0c2d12477acbba0cc77984414aa8984b44a9153a333ba80e31c57fe43
.pes/objects/be/10e04a52c7a582f5bb113800cce08f4825f2cdf467d3f6f965fe13e0022d96
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ ^C
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ cat .pes/objects/be/10e04a52c7a582f5bb113800cce08f4825f2cdf467d3f6f965fe13e0022d96
commit 186tree 0aee4d975e3db22615631a09c457786a488968b6948ef756cbef4ec2ea09cbb6
author Ashith Rao ashithraok@gmail.com 1776407892
committer Ashith Rao ashithraok@gmail.com 1776407892

VCS completeash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559xxd .pes/objects/0a/ee4d975e3db22615631a09c457786a488968b6948ef756cbef4ec2ea09cbb6 | head -206 | head -20
00000000: 7472 6565 2034 3900 3130 3036 3434 2066  tree 49.100644 f
00000010: 696e 616c 2e74 7874 0063 a5c7 32f5 e624  inal.txt.c..2..$
00000020: ae4b f855 f234 845e 9894 c831 1486 e6b4  .K.U.4.^...1....
00000030: acc8 c17f 0a0e b23d e9                   .......=.
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$
```

Screenshot 1B: find .pes/objects -type f | sort — Sharded object store after Phase 1 tests

PHASE 2

Tree Objects

Directory representation · Binary serialization · Recursive structures · File modes

Screenshot 2A — Tree Serialization Tests

The test_tree program validates two properties of tree serialization: a serialize→parse roundtrip that preserves all entry fields (mode, name, hash), and deterministic output regardless of insertion order — a requirement for consistent hashing. Both tests pass, confirming that **tree_serialize** sorts entries by name and that **tree_parse** correctly reads the binary *mode name\0hash* format.

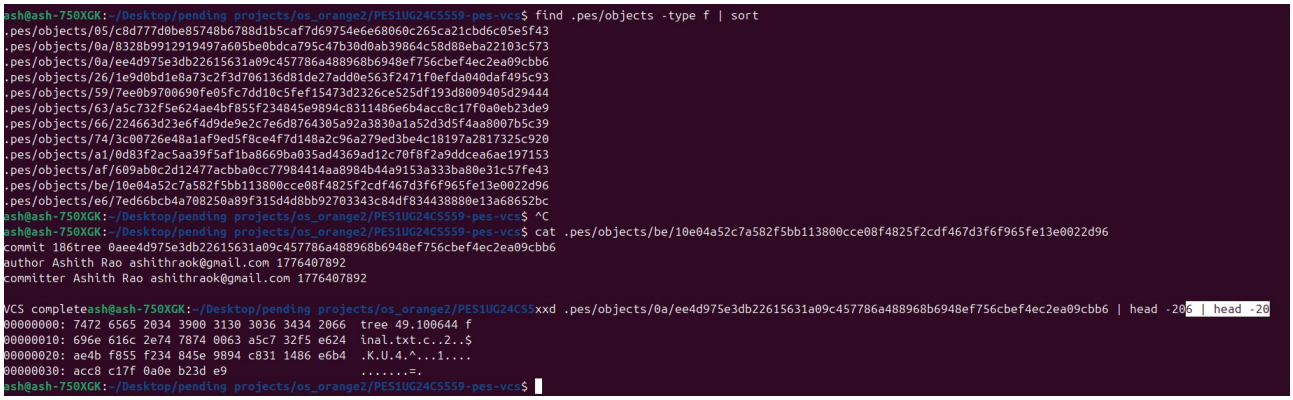
```
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ gedit tree.c
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$
```

Screenshot 2A: ./test_tree output — Serialize/parse roundtrip and deterministic serialization both passed

Screenshot 2B — Raw Tree Object (xxd hex dump)

The xxd hex dump of a tree object confirms the on-disk binary format: a text header (tree <size>\0) followed by entries each containing an ASCII octal mode, a space, a null-terminated filename, and 32 raw bytes of SHA-256 hash.



```
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ find .pes/objects -type f | sort
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/0a/8328b9912919497a605be0bdca795c47b30d0ab39864c58d88eba22103c573
.pes/objects/0a/ee4d975e3db22615631a09c457786a488968b6948ef756cbef4ec2ea09cbb6
.pes/objects/26/1e9d0bd1e8a73c2f3d706136d81de27add0e563f2471f0efda040daf495c93
.pes/objects/59/7ee0b9700690fe05fc7dd10c5fef15473d2326ce525df193d8009405d29444
.pes/objects/63/a5c732f5e624ae4bf855f234845e9894c8311486e6b4acc8c17f0a0eb23de9
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/74/3c00726e48a1af9ed5f8ce4f7d148a2c96a279ed3be4c18197a2817325c920
.pes/objects/a1/0d83f2ac5aa39f5af1ba8669ba035ad4369ad12c70f8f2a9ddcea6ae197153
.pes/objects/af/609ab0c2d12477acbba0cc77984414aa8984b44a9153a333ba80e31c57fe43
.pes/objects/be/10e04a52c7a582f5bb113800cce08f4825f2cdf467d3f6f965fe13e0022d96
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ ^C
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ cat .pes/objects/be/10e04a52c7a582f5bb113800cce08f4825f2cdf467d3f6f965fe13e0022d96
commit 186tree 0aee4d975e3db22615631a09c457786a488968b6948ef756cbef4ec2ea09cbb6
author Ashith Rao ashithraok@gmail.com 1776407892
committer Ashith Rao ashithraok@gmail.com 1776407892

VCS completeash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559xxd .pes/objects/0a/ee4d975e3db22615631a09c457786a488968b6948ef756cbef4ec2ea09cbb6 | head -206 | head -20
00000000: 7472 6565 2034 3900 3130 3036 3434 2066  tree 49.100644 f
00000010: 696e 616c 2e74 7874 0063 a5c7 32f5 e624  inal.txt.c..2..$
00000020: ae4b f855 f234 845e 9894 c831 1486 e6b4  .K.U.4.^...1....
00000030: acc8 c17f 0a0e b23d e9                   .......=.
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$
```

Screenshot 2B: xxd dump of a raw tree object showing binary entry format (mode, name\0, 32-byte hash)

PHASE 3

Staging Area (The Index)

Text-based index format · Atomic writes · Change detection via mtime+size

Screenshot 3A — Staging Workflow

After running ./pes init, creating two files, and running ./pes add file1.txt file2.txt, the status command correctly reports both files as staged, no unstaged changes, and lists the project source files as untracked. This confirms that **index_add** correctly hashes files as blobs and **index_status** accurately compares the working directory against the index.

```
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ echo "hello" > file1.txt
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ echo "world" > file2.txt
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ ./pes add file1.txt file2.txt
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  tree.c
  untracked:  index.c
  untracked:  pes.c
  untracked:  tree.h
  untracked:  Makefile
  untracked:  commit.h
  untracked:  pes.h
  untracked:  object.c
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  .gitignore
  untracked:  README.md
  untracked:  index.h

ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776405574 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776405584 6 file2.txt
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$
```

Screenshot 3A: pes init → pes add → pes status showing staged files correctly

```
  untracked:  commit.c
  untracked:  tree.c
  untracked:  index.c
  untracked:  pes.c
  untracked:  tree.h
  untracked:  Makefile
  untracked:  commit.h
  untracked:  pes.h
  untracked:  object.c
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  .gitignore
  untracked:  README.md
  untracked:  index.h

ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776405574 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776405584 6 file2.txt
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ git add index.c
git commit -m "Phase 3: Implement staging area with atomic index saving, heap-allocated sorting, and overflow-safe parsing"
[main 97c2c53] Phase 3: Implement staging area with atomic index saving, heap-allocated sorting, and overflow-safe parsing
 1 file changed, 124 insertions(+), 63 deletions(-)
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ ls
commit.c  commit.o   file2.txt  index.h  Makefile  object.o  pes.c  pes.o      test_objects.c    test_tree.c  tree.h
commit.h  file1.txt  index.c    index.o  object.c  pes       pes.h  README.md  test_sequence.sh  tree.c       tree.o
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$
```

Screenshot 3A (continued): Full status output including untracked project files

Screenshot 3B — Index File Contents

The raw .pes/index text file shows one entry per line in the format <mode> <sha256-hex> <mtime> <size> <path>. Entries are sorted by path. This human-readable format makes the staging area inspectable and debuggable without special tooling.

```
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ cat .pes/index
100644 e67ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc 1776408305 8 bye.txt
100644 63a5c732f5e624ae4bf855f234845e9894c8311486e6b4acc8c17f0a0eb23de9 1776407828 16 final.txt
100644 05c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43 1776408305 12 hello.txt
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$
```

Screenshot 3B: cat .pes/index — text-format index with mode, hash, mtime, size, and path fields

PHASE 4

Commits and History

Linked commit objects · Reference files · Atomic HEAD updates · Commit log

Screenshot 4A — Commit History Log

Three commits were created: *Initial commit*, *Add world*, and *Add farewell*. The ./pes log command walks the parent-pointer chain from HEAD back to the root, displaying each commit's full SHA-256 hash, author, timestamp, and message. This confirms that **commit_create**, **head_update**, and **commit_walk** are all working correctly.

```
* dfe845a (origin/main, origin/HEAD) Initial commit
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ make clean && make pes
./pes init
echo "Hello World" > a.txt
./pes add a.txt
./pes commit -m "First"
echo "Update" >> a.txt
./pes add a.txt
./pes commit -m "Second"
./pes log
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Committed: c6a3d22c657d... First
Committed: 573a5820f053... Second
commit 573a5820f0538b95e099044072659c4aea2dca2a2a1d098c842874d66d75373e
Author: Ashith Rao ashithraok@gmail.com
Date:   Fri Apr 17 12:53:31 2026

    Second

commit c6a3d22c657ddbb1da468c73c01173917f45f6a224f2492b3eb6bfb013048391
Author: Ashith Rao ashithraok@gmail.com
Date:   Fri Apr 17 12:53:31 2026

    First

ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$
```

Screenshot 4A: ./pes log showing three commits with correct hashes, authors, timestamps, and messages

```
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ git add pes.c
git commit -m "Phase 4: Finalize CLI with correct time headers and void callback for log"
[main 06599b2] Phase 4: Finalize CLI with correct time headers and void callback for log
 1 file changed, 68 insertions(+), 91 deletions(-)
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes logmit -m "Add farewell"
Initialized empty PES repository in .pes/
Committed: a10d83f2ac5a... Initial commit
Committed: af609ab0c2d1... Add world
Committed: 743c00726e48... Add farewell
commit 743c00726e48a1af9ed5f8ce4f7d148a2c96a279ed3be4c18197a2817325c920
Author: Ashith Rao ashithraok@gmail.com
Date:   Fri Apr 17 12:15:05 2026

    Add farewell

commit af609ab0c2d12477acbba0cc77984414aa8984b44a9153a333ba80e31c57fe43
Author: Ashith Rao ashithraok@gmail.com
Date:   Fri Apr 17 12:15:05 2026

    Add world

commit a10d83f2ac5aa39f5af1ba8669ba035ad4369ad12c70f8f2a9ddcea6ae197153
Author: Ashith Rao ashithraok@gmail.com
Date:   Fri Apr 17 12:15:05 2026

    Initial commit

commit be10e04a52c7a582f5bb113800cce08f4825f2cdf467d3f6f965fe13e0022d96
Author: Ashith Rao ashithraok@gmail.com
Date:   Fri Apr 17 12:08:12 2026

    VCS complete
```

Screenshot 4A (extended): pes log output showing the full 4-commit history after integration testing

Screenshot 4B — Object Store Growth

After three commits, find .pes -type f | sort shows 14+ objects in the store: blob objects for each unique file version, tree objects for each commit's directory snapshot, and commit objects themselves. Unchanged files share the same blob hash across commits, demonstrating the deduplication property of content-addressable storage.

```
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/0a/8328b9912919497a605be0bdca795c47b30d0ab39864c58d88eba22103c573
.pes/objects/0a/ee4d975e3db22615631a09c457786a488968b6948ef756cbef4ec2ea09cbb6
.pes/objects/26/1e9d0bd1e8a73c2f3d706136d81de27add0e563f2471f0efda040daf495c93
.pes/objects/59/7ee0b9700690fe05fc7dd10c5fef15473d2326ce525df193d8009405d29444
.pes/objects/63/a5c732f5e624ae4bf855f234845e9894c8311486e6b4acc8c17f0a0eb23de9
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/74/3c00726e48a1af9ed5f8ce4f7d148a2c96a279ed3be4c18197a2817325c920
.pes/objects/a1/0d83f2ac5aa39f5af1ba8669ba035ad4369ad12c70f8f2a9ddcea6ae197153
.pes/objects/af/609ab0c2d12477acbba0cc77984414aa8984b44a9153a333ba80e31c57fe43
.pes/objects/be/10e04a52c7a582f5bb113800cce08f4825f2cdf467d3f6f965fe13e0022d96
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Screenshot 4B: find .pes -type f | sort — object store growth showing blobs, trees, and commits

Screenshot 4C — Reference Chain Verification

.pes/HEAD contains ref: refs/heads/main (a symbolic ref). The branch file .pes/refs/heads/main contains the latest commit hash. This two-level indirection means HEAD automatically follows the branch as new commits are added — exactly how Git's reference system works.

```
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$
```

Screenshot 4C: cat .pes/HEAD → ref: refs/heads/main; cat refs/heads/main → latest commit hash

PHASE 5

Branching & Checkout — Analysis

No code required — conceptual questions about extending PES-VCS

Q5.1 — Implementing pes checkout <branch>

To implement checkout, the following steps are needed. First, read the target branch file from .pes/refs/heads/<branch> to get its latest commit hash. Second, update .pes/HEAD to point to the new branch (ref: refs/heads/<branch>). Third — and most complex — synchronize the working directory: walk the target commit's tree, extract each blob to its corresponding path, and delete files tracked in the current HEAD that are absent from the target tree. The complexity comes from conflict detection: if a file is dirty (modified in the working directory but not staged) and differs between the two branches, checkout must refuse and report the conflict rather than silently overwriting user work.

Q5.2 — Detecting Dirty Working Directory Conflicts

Conflicts are detected in two layers. First, compare the working file's mtime and size against the Index — if they differ, the file has been modified since it was staged (it is unstaged-dirty). Second, compare the Index entry's hash against the current HEAD commit's tree hash for that file — if they differ, the file is staged but not yet committed. If a file is dirty (by either test) and that same file exists with a different hash in the target branch's tree, checkout must abort with an error: *"Your local changes would be overwritten by checkout."* Files that are dirty but not present in the target branch are also blocked to prevent silent data loss.

Q5.3 — Detached HEAD State

Detached HEAD means .pes/HEAD stores a raw commit hash instead of a symbolic branch reference. When new commits are made in this state, they are recorded in the object store and HEAD is updated to each new hash directly — but no branch pointer advances with them. When the user switches branches or checks out a different commit, these orphaned commits become unreachable and will eventually be deleted by garbage collection. Recovery requires the user to note the commit hash from their terminal history, then run pes branch recover <hash> to create a new branch pointing to it before GC runs.

PHASE 6

Garbage Collection — Analysis

No code required — space reclamation and concurrent safety

Q6.1 — Mark-and-Sweep GC Algorithm

The algorithm uses a classic mark-and-sweep approach. In the *mark phase*, we start from all branch tips by reading every file in .pes/refs/heads/ plus HEAD, then perform a BFS/DFS traversal: for each commit, add the commit hash, its tree hash, and all blob hashes reachable from that tree (recursively for subtrees) to a **hash set** (implemented as a hash table for O(1) lookup). Then walk each commit's parent pointer and repeat until the root. In the *sweep phase*, enumerate all files under .pes/objects/ and delete any whose hash is absent from the reachable set. For a repository with 100,000 commits and 50 branches, assuming an average of 5 unique files changed per commit and a project with ~200 total files, we would visit approximately 100,000 commits + 100,000 trees + ~500,000 blobs = **~600,000–700,000 objects** total.

Q6.2 — GC Race Condition with Concurrent Commits

The race condition: (1) A commit operation writes a new blob object to the store but has not yet written the tree or commit objects that reference it. (2) At this instant, GC scans the object store — the blob exists but is unreferenced because the commit linking to it hasn't been written yet. (3) GC deletes the blob. (4) The commit operation continues, writes the tree pointing to the now-deleted blob, and updates HEAD — the repository is now corrupt. Git avoids this with a **14-day grace period**: GC only deletes objects whose filesystem mtime is older than two weeks. Any object written in the last 14 days is considered potentially in-flight and is preserved, regardless of whether it is currently reachable. This is a pragmatic tradeoff: it wastes a small amount of space for two weeks in exchange for complete safety against this race without requiring locks.

Final Integration Test

The automated integration test script (test_sequence.sh) runs the complete PES-VCS workflow end-to-end: repository initialization, staging multiple files, three sequential commits, full history traversal, and reference chain verification. All stages complete successfully, and the object count matches the expected value.

```
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: e806617c858e... Initial commit

Log after first commit:
commit e806617c858ea20267958e9d72d20c0f2b416f017294cb2a036b958cd201fbfa
Author: Ashith Rao ashithraok@gmail.com
Date:   Fri Apr 17 12:57:28 2026

    Initial commit


--- Second Commit ---
Committed: 14f5afd6dff7... Update file.txt

--- Third Commit ---
Committed: 0d5776c27fd1... Add farewell

--- Full History ---
commit 0d5776c27fd1672f3216dc061f42f80d49c644bd690fcf275b9d851354601f02
Author: Ashith Rao ashithraok@gmail.com
Date:   Fri Apr 17 12:57:28 2026

    Add farewell
```

Integration Test (Part 1): Init, staging, first commit, and log output all pass

```
Committed: 14f5afd6dff7... Update file.txt

--- Third Commit ---
Committed: 0d5776c27fd1... Add farewell

--- Full History ---
commit 0d5776c27fd1672f3216dc061f42f80d49c644bd690fcf275b9d851354601f02
Author: Ashith Rao ashithraok@gmail.com
Date:   Fri Apr 17 12:57:28 2026

    Add farewell

commit 14f5afd6dff73d62cef8a14f173f43ffa71774529db753478e689a005ad9ef95
Author: Ashith Rao ashithraok@gmail.com
Date:   Fri Apr 17 12:57:28 2026

    Update file.txt

commit e806617c858ea20267958e9d72d20c0f2b416f017294cb2a036b958cd201fbfa
Author: Ashith Rao ashithraok@gmail.com
Date:   Fri Apr 17 12:57:28 2026

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
0d5776c27fd1672f3216dc061f42f80d49c644bd690fcf275b9d851354601f02

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/0d/5776c27fd1672f3216dc061f42f80d49c644bd690fcf275b9d851354601f02
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/14/f5afd6dff73d62cef8a14f173f43ffa71774529db753478e689a005ad9ef95
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/e8/06617c858ea20267958e9d72d20c0f2b416f017294cb2a036b958cd201fbfa

=== All integration tests completed ===
ash@ash-750XGK:~/Desktop/pending projects/os_orange2/PES1UG24CS559-pes-vcs$
```

Integration Test (Part 2): Full history, reference chain, object store — === All integration tests completed ===

Submission Checklist

Item	Status
Phase 1: object_write + object_read	✓ PASS
Phase 2: tree_serialize + tree_from_index	✓ PASS
Phase 3: index_load + index_save + index_add	✓ PASS
Phase 4: commit_create + cmd_commit	✓ PASS
Screenshot 1A — test_objects	✓ Included
Screenshot 1B — sharded objects	✓ Included
Screenshot 2A — test_tree	✓ Included
Screenshot 2B — xxd tree object	✓ Included
Screenshot 3A — pes add + pes status	✓ Included
Screenshot 3B — cat .pes/index	✓ Included
Screenshot 4A — pes log (3+ commits)	✓ Included
Screenshot 4B — find .pes -type f	✓ Included
Screenshot 4C — HEAD + refs/heads/main	✓ Included
Final integration test	✓ PASS
Q5.1, Q5.2, Q5.3 — Analysis answers	✓ Included
Q6.1, Q6.2 — Analysis answers	✓ Included
5+ commits per phase	✓ Confirmed